

RESEARCH ARTICLE **OPEN ACCESS**

Qubit-Efficient Quantum Local Search for Combinatorial Optimization

Mikhail Podobrii  | Viacheslav Kuzmin  | Vladimir Voloshinov  | Margarita Veshchezerova  | Michael R. Perelshtein 

Terra Quantum AG, St. Gallen, Switzerland

Correspondence: Mikhail Podobrii (mpd@terraquantum.swiss)

Received: 10 June 2025 | **Revised:** 29 January 2026 | **Accepted:** 2 February 2026

Keywords: combinatorial optimization | local search | qubit-efficient encoding | variational quantum algorithm

ABSTRACT

An essential component of many sophisticated metaheuristics for solving combinatorial optimization problems is some variation of a local search routine that iteratively searches for a better solution within a chosen set of immediate neighbors. The size l of this set is limited due to the computational costs required to run the method on classical processing units. We present a qubit-efficient variational quantum algorithm that implements a quantum version of local search with only $\lceil \log_2 l \rceil$ qubits and, therefore, can potentially work with classically intractable neighborhood sizes. Increasing the amount of quantum resources employed in the algorithm allows for a larger neighborhood size, improving the quality of obtained solutions. This trade-off is crucial for present and near-term quantum devices characterized by a limited number of logical qubits. Numerically simulating our algorithm, we successfully solved the largest graph coloring instance that was tackled by a quantum method. This achievement highlights the algorithm's potential for solving large-scale combinatorial optimization problems on near-term quantum devices.

1 | Introduction

Local search is one of the most fundamental heuristic algorithms successfully applied to many challenging combinatorial optimization problems. The so-called r -local search (or r -flip) algorithm [1] starts from a candidate solution and iteratively explores a chosen neighborhood, of size l , of solutions that differ by at most r bits (or Hamming distance r). The process stops when all the neighbors are not better than the current solution. Many metaheuristics, such as simulated annealing [2], tabu search [3], and memetic algorithms [4], are based on local search [5]. Increasing the size of the neighborhood by increasing r can yield better solutions at the cost of an exponentially growing runtime $l \sim C_n^r \sim O(n^r)$, where C_n^r is the binomial coefficient and n is the total number of variables. Therefore, the parameter r typically must be chosen as small.

In this work, we address the aforementioned problem by introducing a Variational Quantum Algorithm (VQA) that uses a parametrized quantum-circuit ansatz of sufficient depth to implement the r -local search heuristic. The distinct feature of our approach is that the number, N_q , of employed qubits adapts to the chosen parameter r , ranging from $N_q = \lceil \log_2 n \rceil$ when $r = 1$ to $N_q = n$ when $r = n$. In contrast to the previously proposed quantum local search method [6, 7], which reduces the number of qubits by breaking the problem into smaller subproblems, our approach addresses the entire problem directly and achieves qubit reduction through amplitude encoding. That is, for a chosen value of r , our method encodes the probabilities of flipping groups of bits, each involving no more than r bits, into the square norm of the quantum state's amplitudes. Then, the parameters of the variational circuit are tuned so that the probabilities from the resulting quantum state minimize a designed non-linear objective function whose local minima coincide with the local

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial](https://creativecommons.org/licenses/by-nc/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited and is not used for commercial purposes.

© 2026 The Author(s). *Advanced Quantum Technologies* published by Wiley-VCH GmbH

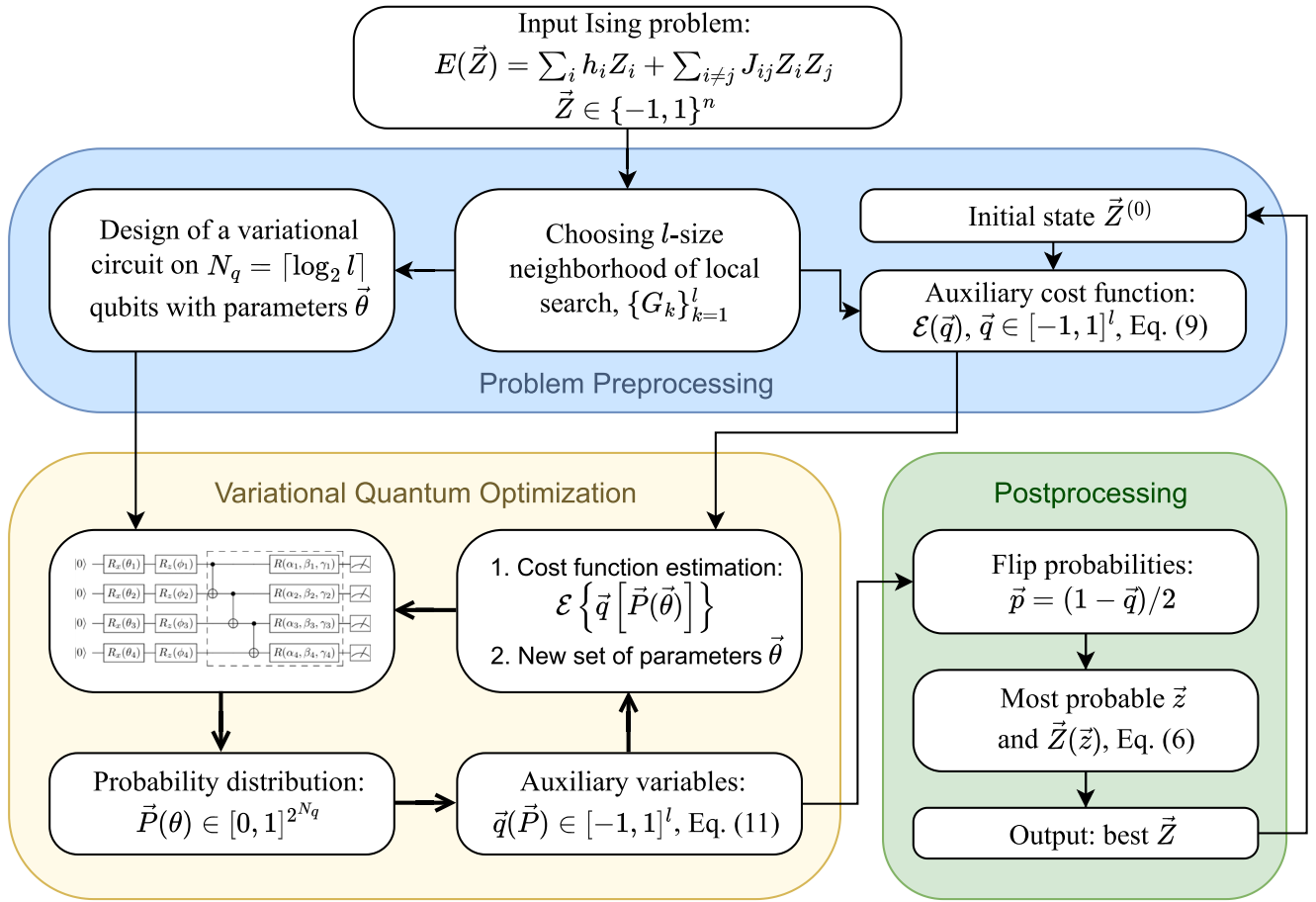


FIGURE 1 | Scheme of our approach. We reduce the initial (discrete) Ising ground state problem to the optimization of a continuous auxiliary function $\mathcal{E}(\vec{q})$ (Equation (9)). The auxiliary function is defined by choosing an initial state $\vec{Z}^{(0)}$ and groups $\{G_k\}$, where $G_k \subseteq [0:n-1]$. To find approximate local minima of the auxiliary function, we employ a variational quantum algorithm, enabling function evaluation even when l is classically intractable. This is achieved by mapping the probability distribution $\vec{P}$ generated from quantum state measurements to the auxiliary variables $\vec{q}$ using Equation (11). After optimization of the quantum circuit, we estimate $\vec{p} = (1 - \vec{q})/2$, which represents the probabilities of group flips. Using these probabilities, we sample several most probable $\vec{z}$, where $z_k = -1$ if the corresponding group G_k is flipped and *one* otherwise. We decode $\vec{z}$ into the original variables $\vec{Z}$ via Equation (6) and choose the best $\vec{Z}$ with the minimal energy. Then, we restart the procedure from the beginning taking $\vec{Z}^{(0)} = \vec{Z}^{\text{best}}$. We perform several such rounds until the solution is not improved.

minima of r -local search. Finally, the optimized quantum state decodes into a set of multi-bit flips that are applied to the initial candidate to generate the resulting solution. We emphasize that our variational quantum algorithm can handle an intractable for classical local search number of neighbors. The scheme of our algorithm is depicted in Figure 1.

Previously, qubit-efficient approaches that require only $N_q = \lceil \log_2 n \rceil$ qubits for solving n -variable optimization problems have been proposed in Refs. [8–12]. However, such an efficient encoding automatically implies that the complexity of classically simulating the involved quantum circuit scales only polynomially with the problem size, limiting the opportunities for achieving quantum advantage on actual quantum hardware. Moreover, in our work, we show that the performance of the method proposed in Refs. [8, 9], referred to as minimal encoding, is limited by the performance of 1-local search due to the presence of multiple local minima in the underlying cost function. While these functions can somehow be optimized with global optimization routines such as differential evolution [13], gradient-based methods will

inevitably fall into local minima. In contrast, while increasing neighborhood size l by employing a growing number, N_q , of qubits, our approach allows us to gradually eliminate these local minima. This leads to higher-quality solutions when the quantum variational ansatz is expressive enough.

At the extreme case, $r = n$, our algorithm requires $N_q = n$ qubits. In this case, all solutions are neighbors to each other, and all the local minima in the cost function are caused by the finite-depth circuit ansatz. This limit is similar to algorithms that use the so-called complete encoding [8], such as VQE [14, 15] and QAOA [16, 17], where the number of qubits is equal to the number of classical variables. In this work, we show that at this limit, our method gives compatible results to VQE with complete encoding, see Appendix A.

Thus, our method intends to fill the gap between the *minimal* and complete encodings. In this intermediate regime, the quantum circuit acts on a substantially larger number of qubits than in the minimal encoding and may therefore no longer be efficiently

simulatable using generic classical simulation methods. At the same time, for a given problem, our method allows the required number of qubits to be adjusted depending on the quantum device at hand. Some other approaches [18, 19] reduce the qubit number by only a constant factor. Reducing the number of qubits in a low-depth variational circuit could also improve the quality of solutions since the cost function of shallow circuits that have many qubits can suffer from severe under-parametrization, see Appendix A.

Other methods with an adaptive number of qubits were previously proposed in Refs. [8, 20, 21]. However, Ref. [8] shows the advantage in solution quality relative to minimal encoding only on sparse problems, while for dense problems, it demonstrates even worse results than minimal encoding due to the presence of contradictions in the procedure for decoding the solution from the quantum state. Furthermore, there is no explanation for how increasing the qubit number affects the quality of the obtained solutions. In contrast, our method provides decoding without any contradictions. We show that it is able to reach the same solution quality as r -local search for $r > 1$ and, therefore, that it outperforms minimal encoding both on sparse and dense problems.

While we were developing our approach, Sciorilli et al. [21] proposed an alternative quantum algorithm for solving combinatorial optimization problems using an adaptive number of qubits, demonstrating competitive with the state-of-the-art classical solvers results on the MaxCut problems. Our method offers the possibility of constructing problem-specific objective functions. This advantage might be crucial for sparse and constrained problems. This is demonstrated in our numerical experiment on the graph coloring problem, where our approach successfully solves the largest instance tackled by a quantum algorithm to date.

The structure of this paper is organized as follows. In Section 2, we begin by introducing an auxiliary objective function whose local minima match those of local search over a specified neighborhood and discuss an efficient selection of the appropriate neighborhood for a given problem. Next, in Section 3, we outline how to optimize the auxiliary function using a variational quantum algorithm. In Section 4, we analyze the resource requirements to show that the quantum algorithm can handle an intractable for classical local search number of neighbors. In Section 5, we specify the simulation details and the considered problems. Finally, in Section 6, we present examples demonstrating our algorithm's competitive performance across various problems on a simulator and on a real IBM quantum device.

2 | Objective Function

In our work, we focus on a class of quadratic unconstrained binary optimization (QUBO) problems, which is equivalent to the class of Ising models. These classes are general since most combinatorial optimization problems can be formulated as QUBO or Ising problems [22, 23].

QUBO problems are formulated as follows: for a given real upper-triangular matrix $A \in \mathbb{R}^{n \times n}$, find a binary vector $\vec{x}^* \in \{0, 1\}^n$ that

minimizes the function:

$$C(\vec{x}) = \vec{x}^T A \vec{x} = \sum_{i,j=1}^n a_{ij} x_i x_j = \sum_i a_{ii} x_i + \sum_{i<j} a_{ij} x_i x_j \quad (1)$$

By transforming $x_i \rightarrow (1 - Z_i)/2$, QUBO problems can be reduced to the Ising model:

$$E(\vec{Z}) = \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j \quad (2)$$

where $Z_i \in \{-1, 1\}$ are spin variables. The coefficients are related as $J_{ij} = a_{ij}/4$, $h_i = -\sum_k (a_{ik} + a_{ki})/4$.

The original problem, given by Equation (1) or (2), is defined on a discrete space. In the following, we define an auxiliary problem over continuous variables that can be optimized with a variational quantum algorithm and whose solution is equivalent to the original problem.

2.1 | Bilinear Relaxation

The bilinear continuous relaxation of the problems given by Equations (1) and (2) are:

$$\text{QUBO: } \bar{C}(\vec{p}) = \sum_i a_{ii} p_i + \sum_{i<j} a_{ij} p_i p_j, \quad p_i \in [0, 1] \quad (3)$$

$$\text{Ising: } \bar{E}(\vec{q}) = \sum_i h_i q_i + \sum_{i<j} J_{ij} q_i q_j, \quad q_i \in [-1, 1] \quad (4)$$

Assuming that x_i are independent random variables, the continuous function Equation (3) is precisely [24] the average value of the original discrete objective function Equation (1) over a variable assignment distribution, where $p_i = P(x_i = 1)$ is the probability that x_i has value 1. For the Ising model, consequently, $(1 - q_i)/2$ is the probability $P(Z_i = -1)$.

The functions (3) and (4) have the same minimum as Equations (1) and (2) over discrete variables [24]; therefore, optimization of the bilinear relaxation over the corresponding cube $([0, 1]^n$ for QUBO or $[-1, 1]^n$ for Ising) is equivalent to the original discrete problem.

We analyze the local minima to assess the solution quality achieved by optimizing the bilinear relaxation using gradient-based methods. The following propositions characterize the local minima of multilinear functions on the hypercube:

Proposition 1. *Let $f(\vec{p})$ be a multilinear function. Then, on the hypercube $[a, b]^n$, all local minima are at its vertices - points in $\{a, b\}^n$.*

The proof of this proposition is given in Ref. [25]. As an example, it means that for function Equation (3), the local minima reside in $\{0, 1\}^n$.

Proposition 2. *Let $\bar{a} := b$, $\bar{b} := a$ (e.g. for $x \in \{0, 1\}$ the $\bar{x}$ is logical negation). $\vec{p} \in \{a, b\}^n$ is a local minimum of f over $[a, b]^n \iff f(p_1, \dots, \bar{p}_i, \dots, p_n) > f(p_1, \dots, p_i, \dots, p_n)$ for all $i \in [1:n]$*

Proof. Consider the vertex $A = (a, \dots, a)$ without loss of generality. A is a local minimum if and only if $\partial f / \partial p_i > 0$ for all i . Since a multilinear function is linear by each variable when others are fixed:

$$\frac{\partial f}{\partial p_i}(A) = \frac{f(a, \dots, p_i = b, \dots, a) - f(a, \dots, p_i = a, \dots, a)}{b - a} \quad (5)$$

Given that $b > a$, we obtain the desired statement. $\square$

Applying the proposition two to Equation (3), we obtain that a solution is a local minimum of the bilinear relaxation if and only if it is better than all neighbor solutions differing by one bit. Note that this condition coincides with the local minimum of one-local search. A gradient-based optimizer may get stuck in any local minimum; therefore, optimization of function in Equation (3) gives solutions of similar quality as 1-local search. As a consequence, the results can be very poor: for some constrained optimization problems, such as the traveling salesman problem or graph coloring, when the constraints are treated as penalties, every feasible solution is a local optimum for one-local search.

2.2 | Encoding Larger Neighborhoods

In bilinear relaxation, each variable encodes a single bit of the solution. As a result, the set of neighbors in the corresponding local search consists of only solutions that differ by one bit. To include more neighbors in the set, we add variables that control flipping groups of several bits.

Starting from here, we focus on the Ising representation of the QUBO problem, for which the calculations are more straightforward. First, we choose the initial state of the spins to be $\vec{Z}^{(0)}$. Then, we introduce a new set of l variables, $z_k \in \{-1, 1\}$, where each variable corresponds to a flip of a specific subset G_k of spins. States of the original Ising variables are decoded back by the following product:

$$Z_i = Z_i^{(0)} \prod_{k: i \in G_k} z_k \quad (6)$$

which counts how many times i -th spin is flipped within the groups. This definition implies that flipping z_k results in flipping all spins Z_i in G_k .

Substituting Equation (6) into Equation (2), we obtain:

$$E(\vec{z}) = \sum_i h_i Z_i^{(0)} \prod_{k: i \in G_k} z_k + \sum_{i < j} J_{ij} Z_i^{(0)} Z_j^{(0)} \prod_{k: i \in G_k} z_k \prod_{k: j \in G_k} z_k \quad (7)$$

Since $z_k^2 = 1$, Equation (7) can be rewritten in a multilinear form:

$$E(\vec{z}) = \sum_i h_i Z_i^{(0)} \prod_{k: i \in G_k} z_k + \sum_{i < j} J_{ij} Z_i^{(0)} Z_j^{(0)} \prod_{k: i \in G_k} z_k \prod_{k: i \notin G_k} z_k \prod_{k: j \in G_k} z_k \quad (8)$$

As with bilinear relaxation, this allows the discrete variables $z_k \in \{-1, 1\}$ to be replaced with continuous variables $q_k \in [-1, 1]$. We call the resulting function the auxiliary function:

$$\mathcal{E}(\vec{q}) = \sum_i h_i Z_i^{(0)} \prod_{k: i \in G_k} q_k + \sum_{i < j} J_{ij} Z_i^{(0)} Z_j^{(0)} \prod_{k: i \in G_k} q_k \prod_{k: i \notin G_k} q_k \prod_{k: j \in G_k} q_k \quad (9)$$

In a similar fashion to the aforementioned bilinear relaxation, Equation (9) represents the average value of the Equation (8) over a variable assignment distribution where z_k are independent and $(1 - q_k)/2$ is the probability of $z_k = -1$.

Applying proposition 2 to Equation (9), one can prove that the local minima of this function correspond to the solutions that are better than all neighboring solutions differing by spin-flips encoded in groups $\{G_k\}$, i.e., we built an analog of a local search over a chosen neighborhood. When groups $G_k = \{k\}$ with $k \in [1:n]$ include only one bit and all $Z_i^{(0)} = 1$, the auxiliary function (Equation (9)) matches with the bilinear relaxation (Equation (4)). If all subsets of spins are encoded, i.e., $\{G_k\} = \mathcal{P}(\{1, 2, \dots, n\})$, where $\mathcal{P}(S)$ denotes a power set of S , then the local minima of the auxiliary function map to the global optima of the initial problem. However, in that case, the number of groups and, consequently, the number of variables in the auxiliary function equals 2^n .

Note that Equation (9) is a high-order polynomial. For instance, for r -flip groups, the degree of the product $\prod_{k: i \in G_k} q_k$ equals $O(n^{r-1})$. Since $q_k \in [-1, 1]$, this leads to a barren plateau-like problem [26], where the Equation (9) has values close to zero for most of the landscape. Moreover, since taking the derivative reduces the degree only by one, this issue also concerns the gradient, hessian, and so on, leading to an adverse flat landscape if r is large enough. We mitigate this problem by an additional reparameterization described in Section 3.2.

The presented approach can be straightforwardly extended to general PUBO problems, as shown in Appendix B.

2.3 | Neighbors Choice

For an optimization problem with all-to-all interactions, there is no general rule for choosing a set of neighbors for a given r -local search; thus, in this scenario, all solutions at Hamming distance at most r can be included in the set of neighbors defined by $\{G_k\}$. However, for sparse problems, it makes sense to select only a problem-specific subset of the neighbors.

Indeed, let's consider a 1-local search or bilinear relaxation. The increment of the energy in Equation (2) after inverting the k -th spin is

$$\Delta_k = -2Z_k(h_k + \sum_{i \neq k} (J_{ki} + J_{ik})Z_i) \quad (10)$$

where Z_k is the spin value in the current solution.

From this follows that Δ_k depends on Z_k and adjacent spins, i.e., those spins Z_i for which $J_{ki} + J_{ik} \neq 0$. Therefore, the increment of the energy after the simultaneous inversion of two nonadjacent spins (m, k) is $\Delta_{mk} = \Delta_m + \Delta_k$. For the local minimum of one-local search or the bilinear relaxation, $\Delta_m > 0$ and $\Delta_k > 0$, and therefore $\Delta_{mk} > 0$. In other words, if neither flipping the m -th spin nor flipping the k -th spin improves the solution, neither will the simultaneous flipping of both of them. That is, if we pursue the elimination of local minima, it makes sense to include only adjacent spin pairs into groups $\{G_k\}$.

The same idea can be generalized to large spin sets: if a group of spins G is a union of disjoint groups G_1, G_2 , where no elements $k \in G_1$ and $m \in G_2$ interact ($J_{km}=J_{mk}=0$), then the increment in the energy after inverting all the spins from G is $\Delta_G = \Delta_{G_1} + \Delta_{G_2}$. If G_1 and G_2 are encoded, encoding G does not contribute to the elimination of the local minima. Therefore, in general, it makes sense to encode only connected induced subgraphs of size $\leq r$ to implement r -local search.

Moreover, for constrained problems, additional groups can be excluded if we can ensure that flipping them in a feasible solution would lead to an infeasible one. An example of this is the graph coloring problem, which we consider in Section 6.3.

3 | Variational Quantum Algorithm

The main result of the previous sections is that we can imitate r -local search for an Ising problem by minimizing the auxiliary function given by Equation (9). However, the number of variables (groups) l grows quickly with r : it varies from $l = n$ for bilinear relaxation to $l = 2^n$ for the case where $\{G_k\}$ are all subsets of spins, in which case all solutions are neighbors of each other. Thus, r is limited by computational capabilities.

In this section, we address this problem by introducing a variational quantum algorithm aimed at finding a minimum of Equation (9). Our approach uses a parametrized quantum circuit to prepare a quantum state and a specially designed algorithm to associate each obtained measurement outcome to some group G_k . The probability of each outcome is converted to the probability of flipping the corresponding group. These flip probabilities can be used for estimating the auxiliary cost function in Equation (9), which can then be minimized by updating the circuit parameters. When the circuit optimization is finished, the flip probabilities are used to sample the most probable solutions.

We acknowledge that the previous analysis of the local minima is no longer applicable due to the fact that the optimization targets the parameters of the quantum circuit rather than $\vec{q}$ directly. Nevertheless, our numerical experiments for small-scale problems (see Section 6.2) demonstrate that if the circuit depth is sufficient, the solutions are similar to those obtained by classical local search over the corresponding neighborhood. Remarkably, we achieve this performance with the number of optimized circuit parameters much fewer than l .

In Section 3.1, we show how to get the correspondence between the measurement outcomes and the groups. In Section 3.2, we introduce the mapping between the outcome probabilities and the flip probabilities. In Section 3.3, we specify the variational ansatz used in our algorithm. Finally, in Section 3.4, we describe the entire workflow.

3.1 | Mapping Measurement Output to Group

Each measurement outcome μ (a bitstring of length N_q) must be decoded into a group $G_\mu \subseteq \{0, 1, \dots, n-1\}$, representing the set of spins to flip in the original problem. In what follows, we describe how to construct this mapping for different neighborhood types.

3.1.1 | Trivial Mapping

When the total number of groups l is modest and can be stored explicitly as a list $\{G_k\}_{k=0}^{l-1}$, we use $\lceil \log_2 l \rceil$ qubits and define G_μ to be the μ -th group in the list for $\mu < l$; outcomes with $\mu \geq l$ are discarded. We refer to this as the trivial mapping. This approach is practical only when l is small enough to be enumerated classically.

For larger neighborhoods, where enumerating all groups is infeasible, we employ implicit representations and efficient decoding routines. We consider two common cases: the full r -neighborhood and the sparse neighborhood consisting only of connected subgraphs.

3.1.2 | Full r -Neighborhood

Here, the groups consist of all subsets of $\{0, \dots, n-1\}$ of size at most r , corresponding to all solutions within Hamming distance r . The total number of such groups is $l = \sum_{m=1}^r \binom{n}{m}$.

A simple decoding method interprets the measurement outcome μ as a base- n integer with up to r digits. The set of distinct digits then defines the group G_μ . For example, with $n = 16$ and a 12-qubit outcome $|010010110011\rangle$ (which equals $4B3_{16}$ in hexadecimal), the distinct digits are 3, 4 and 11, giving $G_\mu = \{3, 4, 11\}$. Because digits may repeat, several different μ can map to the same group; the total number of possible outcomes is n^r . This scheme requires $N_q = \lceil r \log_2 n \rceil$ qubits, which is slightly larger than the information-theoretic minimum $\lceil \log_2 l \rceil$. In the extreme case $r = n$, the overhead is a factor $\log_2 n$. The conversion from μ to the base- n digits can be performed in time $O(\log(r \log n) \cdot M(r \log n))$, where $M(b) = O(b^2)$ is the cost of multiplying two b -bit numbers [27].

An alternative method that avoids the qubit overhead uses the trivial mapping together with an efficient unranking algorithm that directly computes the μ -th subset of size at most r without storing the whole list. This approach is described in Appendix C and requires only $\lceil \log_2 l \rceil$ qubits, at the cost of a higher classical decoding time.

3.1.3 | Sparse Neighborhood

For sparse problems it is often sufficient to consider only groups that form connected subgraphs of the problem's interaction graph, as argued in Section 2.3. Counting all connected subgraphs is #P-complete [28, 29], so mapping without qubit overhead is not available in general case. Nevertheless, we can still decode μ efficiently with a modest qubit number overhead.

Assume the interaction graph is d -regular. Any connected subgraph of size at most r can be described by a starting vertex $s \in \{0, \dots, n-1\}$ and a sequence of $r-1$ choices of neighbouring vertices. Concretely, we represent the group as $W = (s, j_2, \dots, j_r)$ where each $j_p \in \{0, \dots, d-1\}$ indicates which neighbour of the previously chosen vertex is added at step p .

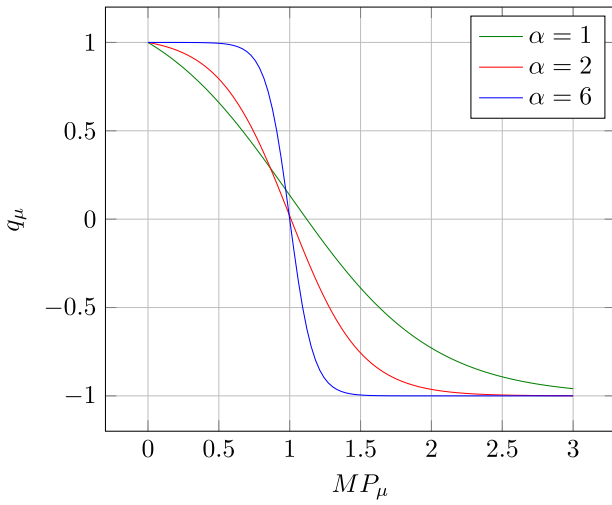


FIGURE 2 | Plots of the transformation $q_\mu(P_\mu)$ for different values of the hyperparameter α .

Given an integer $\mu \in \{0, \dots, nd^{r-1} - 1\}$, we extract the starting vertex as $s = \lfloor \mu/d^{r-1} \rfloor$ and the remaining indices by writing $\mu \bmod d^{r-1}$ in base d . This defines a unique walk W , which is then expanded into the corresponding set of vertices G_μ . The number of qubits required is

$$N_q = \lceil \log_2 n + (r-1) \log_2 d \rceil$$

This construction generalizes straightforwardly to non-regular graphs by using vertex-dependent branching factors; the worst-case overhead remains logarithmic in the graph parameters.

3.2 | Mapping Outcome Probabilities to Variables $\vec{q}$

To map the probabilities $P_\mu \in [0, 1]$ of measurement outcome μ to the auxiliary function variables $q_\mu \in [-1, 1]$, we suggest using the following continuous monotonic transformation:

$$q_\mu = 2 \frac{\tanh(\alpha(1 - MP_\mu)) + 1}{\tanh \alpha + 1} - 1 \quad (11)$$

given in Figure 2, where $\alpha > 0$ and $M > 0$ are additional hyperparameters. The parameter α makes the function more step-like, thereby reducing the adverse area where many $q_\mu \approx 0$, mentioned in Section 2.2. The parameter M impacts how many q_μ can fall in a negative value: under transformation given by Equation (11), it happens only for values $P_\mu \gtrsim 1/M$. Since $P_\mu \in [0, 1]$ and $\sum_\mu P_\mu \leq 1$, not more than M values q_μ can be negative (see Table 1 for an example). In Section 4.2, we show that this limitation provides a good finite measurement estimate of the auxiliary function.

The property $q_\mu(0) = 1$ implies that if the outcome μ was not measured in N rounds, the corresponding variable q_μ gets value *one*. Therefore, our encoding avoids the issue present in minimal encoding [8, 9], where outcomes that did not occur lead to an undefined bit value.

TABLE 1 | We consider a demonstrative probability distribution over a *three*-qubit state $\vec{P} = \{1/4, 1/4, 1/8, 1/8, 1/8, 1/16, 1/16, 0\}$. For different values of α and M , we show the corresponding $\vec{q}$. As can be seen, the parameter M affects the number of negative q_μ : larger values of M result in a larger number of negative q_μ for a given α and $\vec{P}$. When α is small, q_μ changes smoothly from -1 to 1 as P_μ increases; if α is large, q_μ experiences an abrupt change when $P_\mu \approx 1/M$.

		$\vec{P}$							
		1/4	1/4	1/8	1/8	1/8	1/16	1/16	0
M	α	$\vec{q}$							
2	1	0.66	0.66	0.86	0.86	0.86	0.93	0.93	1.00
4	1	0.14	0.14	0.66	0.66	0.66	0.86	0.86	1.00
8	1	-0.73	-0.73	0.14	0.14	0.14	0.66	0.66	1.00
16	1	-0.99	-0.99	-0.73	-0.73	-0.73	0.14	0.14	1.00
2	2	0.79	0.79	0.94	0.94	0.94	0.98	0.98	1.00
4	2	0.02	0.02	0.79	0.79	0.79	0.94	0.94	1.00
8	2	-0.96	-0.96	0.02	0.02	0.02	0.79	0.79	1.00
16	2	-1.00	-1.00	-0.96	-0.96	-0.96	0.02	0.02	1.00
2	3	0.91	0.91	0.98	0.98	0.98	0.99	0.99	1.00
4	3	0.00	0.00	0.91	0.91	0.91	0.98	0.98	1.00
8	3	-1.00	-1.00	0.00	0.00	0.00	0.91	0.91	1.00
16	3	-1.00	-1.00	-1.00	-1.00	-1.00	0.00	0.00	1.00

3.3 | Variational Circuit

We use a common approach to address optimization problems that lack a problem-inspired circuit ansatz. We exploit a hardware-efficient ansatz [30] and use gates that can be efficiently transpiled into the gates available on the IBM quantum device of type *Eagle r3* (<https://docs.quantum.ibm.com/guides/processor-types>). The structure of the ansatz on N_q qubits, given in Figure 3, is chosen empirically. The ansatz consists of Hadamard gates on each qubit and a sequence of L identical layers, consisting of parameterized Z and Y rotations and two-qubit entangling ECR gates, that have the following matrix representation:

$$\text{ECR} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 1 & 0 & i \\ 1 & 0 & -i & 0 \\ 0 & i & 0 & 1 \\ -i & 0 & 1 & 0 \end{pmatrix} \quad (12)$$

The obtained circuit has $2N_q L$ trainable parameters $\vec{\theta}$.

3.4 | Optimization Process

After substituting Equation (11) into Equation (9) and introducing a parameterized quantum circuit, we obtain a composite auxiliary function $\mathcal{E} = \mathcal{E}[\vec{q}(\vec{P}(\vec{\theta}))]$. Here, we outline how to optimize it and get the solution to the original problem.

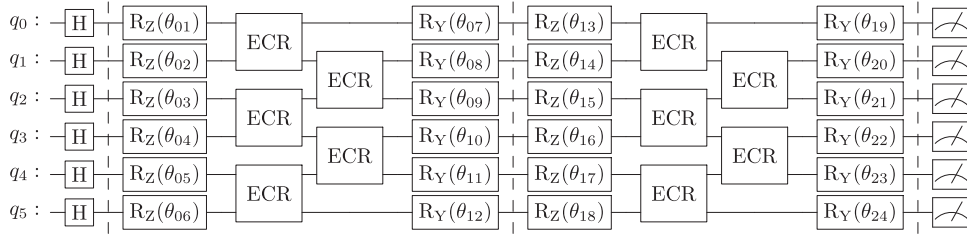


FIGURE 3 | Ansatz with 6 qubits and 2 layers.

3.4.1 | Initial Solution Selection

First, we choose an initial solution by fixing $\vec{Z}^{(0)}$ in Equation (9). Typically, it may be a state with all spins set to one, a random state, or a random feasible solution for constrained problems.

3.4.2 | Hyperparameter Selection

For r -local search, we set $M \sim n/r$, where n is the number of variables in the original problem. This choice ensures that the algorithm can explore a sufficient number of group flips to reach any solution in the neighborhood. For α , we typically use $\alpha \in [2, 10]$, with larger values preferred for larger problems or when r is increased.

3.4.3 | Parameter Optimization

In order to estimate the auxiliary function, the quantum circuit is prepared and sampled N times in a computational basis resulting in the empirical probability distribution $\{N_\mu/N\}$ estimating $\vec{P}$, where N_μ is the number of shots providing the μ -th outcome. Note that although vector $\vec{P}$ is 2^{N_q} -dimensional, its estimate – obtained with N shots – has no more than N nonzero components, and this allows for efficient computations in a sparse format.

To optimize the auxiliary function using gradient-based methods, we also need to compute the gradient. We are interested in

$$\frac{\partial \mathcal{E}}{\partial \vec{\theta}} = \frac{\partial \mathcal{E}}{\partial \vec{q}} \frac{\partial \vec{q}}{\partial \vec{P}} \frac{\partial \vec{P}}{\partial \vec{\theta}} \quad (13)$$

where $\partial \mathcal{E} / \partial \vec{q}$ and $\partial \vec{q} / \partial \vec{P}$ are computed by direct differentiation of Equation (9) and Equation (11). We can compute $\partial \vec{P} / \partial \vec{\theta}$ using the parameter-shift rule [31]:

$$\frac{\partial \vec{P}}{\partial \theta_j} = \frac{1}{2} \left(\vec{P}(\theta_j + \pi/2) - \vec{P}(\theta_j - \pi/2) \right) \quad (14)$$

The optimization process starts with a random initialization of the parameters $\vec{\theta} = \vec{\theta}_0$ from the uniform distribution on $[0, 2\pi)$. The parameters are then updated by some classical optimizer until convergence. After that, the circuit is sampled another N times to obtain an estimate of $\vec{q}$.

3.4.4 | Solution Recovery

We consider every variable $z_k \in \{-1, 1\}$ of the auxiliary problem given by Equation (7) as an independent random variable distributed according to the Bernoulli distribution with “success probability” $p_k = P(z_k = -1) = (1 - q_k)/2$ as shown in Section 2.2. We sample a sequence of the S most probable $\vec{z}$ using the algorithm described in Appendix D and select the one, denoted $\vec{z}^{\text{best}}$, with the minimal energy $E(\vec{z})$ (Equation (7)). Then, we compute the resulting solution for the original problem $\vec{Z}^{\text{best}}$ using definition from Equation (6). We note that there are two different samplings in our algorithm: in the first one, the quantum circuit is sampled to obtain $\vec{p}$ – the parameters of the Bernoulli distribution; in the second one, the solutions are sampled from this distribution.

3.4.5 | Restarts

We remark that $\vec{Z}^{\text{best}}$ found in the previous paragraph is not necessarily a local minimum for the classical local search over a corresponding neighborhood. As previously mentioned, this is due to the fact that the quantum algorithm differs from the direct optimization of the auxiliary function. One limitation of the quantum algorithm arises from the $\vec{q}(\vec{P})$ mapping: as shown in Section 3.2, the resulting $\vec{q}$ contains no more than M negative components. Intuitively, that means the quantum algorithm can’t provide more than $\approx M$ steps of classical local search started from $\vec{Z}^{(0)}$. That is, it makes sense to restart our algorithm taking $\vec{Z}^{(0)} = \vec{Z}^{\text{best}}$. We perform R such rounds, until no improvement in solution quality is observed.

The detailed pseudocode for the complete algorithm is given in Appendix E.

4 | Resource Analysis

In this section, we analyze the amount of resources, both quantum and classical, required for our algorithm.

4.1 | Number of Layers

The quantum algorithm optimizes the circuit parameters $\vec{\theta}$ rather than $\vec{q}$ directly. As a result, it performs optimization over a subspace of possible values of $\vec{q}$, which may result in the appearance of new local minima [32]. Moreover, in our

approach, we encounter the traditional trade-off in variational quantum algorithms between the expressivity of the quantum circuit and the complexity of the parameter optimization. Indeed, an increased number of layers allows us to cover a larger subspace of the domain of variables $\vec{q}$ but increases the difficulty of parameter tuning.

In addition, our scheme gives freedom in choosing the required number of qubits to address the same optimization problem by selecting the number l of groups to encode. Since fewer qubits generally require fewer layers to achieve good expressivity, we face another trade-off: we need to choose l large enough to eliminate local minima of the bilinear relaxation while still allowing the auxiliary function to be optimized with a reasonable number of layers. In Section 6.2, we provide numerical evidence that the number of layers doesn't need to grow linearly with l to achieve good performance, showcasing the advantage of the quantum algorithm over classical optimization of the auxiliary function.

We acknowledge that our algorithm may encounter a barren plateau problem [26] inherent to the hardware-efficient circuit. Since our algorithm can utilize arbitrary parameterized circuits, it may be possible to choose a more sophisticated problem-inspired ansatz, which falls into the scope of further research.

4.2 | Number of Measurements

As explained in Section 3.4, on a real quantum device, the auxiliary function is estimated from a finite number of circuit measurements. In this section, we analyze how many measurements (shots) are required to estimate the auxiliary function with a given accuracy. The inherent difficulty of the auxiliary function is that it is non-linear and can't be expressed as a quantum observable as in VQE and QAOA.

The estimate, $\hat{\mathcal{E}}$, for the true auxiliary function value, $\mathcal{E}$, is obtained by measuring the quantum circuit N times and is equivalent to the value of the auxiliary function at a nearby point, $\hat{P}$. We treat $\hat{\mathcal{E}}$ as a random variable and evaluate the quality of the approximation using the mean squared error, $\text{MSE} = \mathbb{E}(\hat{\mathcal{E}} - \mathcal{E})^2$. The absolute value of the MSE depends both on the number of shots and on the flatness of the function landscape around the evaluation point.

To investigate the number of shots required to obtain a good estimate, we conduct an experiment using a three-regular MaxCut-256 graph with weights randomly assigned in the interval $[-1, 1]$ (see Section 5). For several values $r = 1, 3, 5, 6, 7$, we take a randomly initialized circuit with $L = 10$ layers and the number of qubits corresponding to the chosen r . We compute the exact value $\mathcal{E}$ of the auxiliary function by employing exact state-vector simulation and generate the set of 10000 estimates $\{\hat{\mathcal{E}}_i\}$, where each estimated value is obtained by sampling the quantum circuit N times. Conducting the experiment for N in the range 1 to 2^{24} , we compute the MSE of the estimates as a function of N . We repeat the experiment with a fixed $\alpha = 3$ and various values of $M = 2, 20, 200, 2000$. The results are presented in Figure 4.

The right plot of Figure 4 demonstrates that the MSE starts decreasing only when $N \gtrsim M$. Intuitively, such behavior can be

explained as follows: if $N < M$, then the outcomes that are sampled at least once have an estimated probability $N_\mu/N > 1/M$, which corresponds to $q_\mu < 0$. In that case, the corresponding groups will be flipped even if the true probability P_μ is much smaller and doesn't correspond to a negative q_μ . As a result, when $N < M$, the estimate becomes highly inaccurate, and only $N \gg M$ provides a good estimate. Furthermore, the plot indicates that when N is sufficiently large, the MSE asymptotically scales as $\propto 1/N$. This behavior is similar to complete encoding (see Appendix A).

The left plot of Figure 4 shows the impact of increasing l with a fixed $M = 200$. Remarkably, the behavior does not change significantly as l grows: the MSE still starts decreasing only when $N \gtrsim M$ and reaches the same asymptotic when $N \gg M$. At first glance, one may notice the surprising trend that, for a fixed N , the MSE decreases as l increases. This may be explained as follows: increasing l worsens the flat landscape problem mentioned in Section 2.2. As a result, the variance of the auxiliary function across the landscape decreases, reducing the absolute value of the MSE.

If M is chosen as $M \sim n/r$ (see Section 3.4), the proposed method interpolates continuously between the minimal-encoding regime, where the number of measurements grows exponentially with the number of qubits, and the complete-encoding regime, where the number of shots required for fixed-accuracy estimation does not scale with system size. Under the assumption that the number of measurements scales polynomially with M , as suggested by Figure 4 (right), the resulting measurement complexity is therefore expected to scale at most polynomially with the problem size.

We note, however, that in practice the required number of measurements may increase due to the emergence of barren plateaus [26], which are inherent to hardware-efficient ansätze as the number of qubits grows. This effect can lead to an exponential increase in the number of measurements required for optimization over a flat landscape. Several mitigation strategies have been proposed in the literature (e.g., Ref. [33]), and adapting such techniques to our framework constitutes an important direction for future research. Our contribution shows that, at the level of objective estimation and representation of large neighborhoods, the proposed encoding does not introduce an additional source of exponential overhead beyond what is already known for generic VQAs.

4.3 | Complexity of the Auxiliary Function and Gradient Evaluation

Each product in the auxiliary function given by Equation (9) contains at most l multipliers, and the entire expression includes $O(n^2)$ summands. Therefore, the complexity of evaluating the function does not exceed $O(n^2l)$ in the general case. However, while optimizing with a finite number of measurements, N , there are not more than N nonzero P_μ . Since $q_\mu(P_\mu = 0) = 1$ does not modify the auxiliary function, it can be efficiently computed in at most $O(n^2N)$ classical computing time.

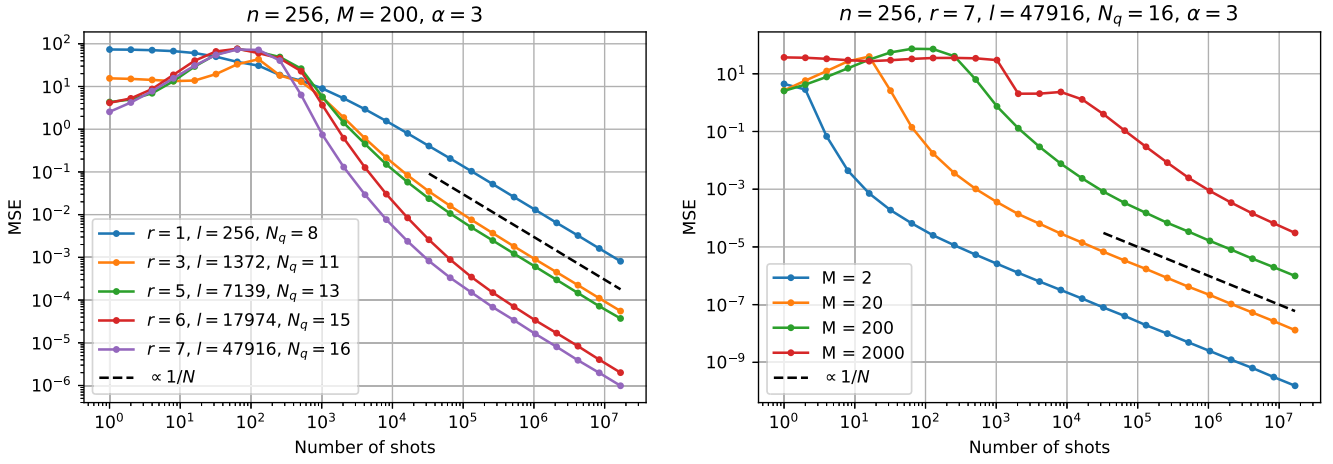


FIGURE 4 | Mean squared error of the auxiliary function estimation obtained with a finite number of measurements (shots). Left: for different r ; right: for different M .

For gradient estimation, the parameter shift rule requires the execution of two circuits for each parameter. Since $\vec{P}$ has no more than N nonzero components, Equation (14) implies that there are no more than $2N$ nonzero components of $\partial \vec{P} / \partial \theta_j$. Each nonzero component requires the evaluation of $\partial q_\mu / \partial P_\mu$, which can be done in $O(1)$, and $\partial \mathcal{E} / \partial q_\mu$, which, similarly to $\mathcal{E}$, requires $O(n^2 N)$ time. As a result, we get $O(n^2 N^2)$ for one parameter and $O(n^2 N^2 L N_q)$ for the entire gradient. Recall that for the trivial mapping, $N_q = O(\log l)$.

It follows from the analysis above that the complexity does not scale linearly with l ; therefore, the auxiliary function and its gradient can be estimated even when classical local search over an l -neighborhood is intractable.

5 | Methods

5.1 | Simulation

For all the numerical experiments described in Section 6, we utilize a noiseless simulator from the `TensorCircuit` [34] library with a Jax backend, which allows for automatic objective function differentiation. All the optimizations, except those in Section 6.4, are performed with a `SciPy` implementation of the L-BFGS-B [35] optimizer with the default settings.

Since we are working with heuristic algorithms that may yield different solutions on each run, we repeat each experiment multiple times to gather statistics. For each run, we choose a random initial solution $\vec{Z}^{(0)}$ and random circuit parameters $\vec{\theta}_0$. The number of groups in our experiments is relatively small, which allows us to store them explicitly and implement the trivial mapping described in Section 3.1.

5.2 | Local Search

For classical local search, we implement the first move improvement strategy [5], where at each step, the first neighbor that is better than the current solution is accepted. The flipping groups that

generate the neighbors are ordered by size, and within the same size, they are ordered lexicographically. For example, if $n = 3$ and $r = 2$, the groups are ordered as $\{0\}, \{1\}, \{2\}, \{0, 1\}, \{0, 2\}, \{1, 2\}$.

5.3 | Problems

5.3.1 | MaxCut

A canonical and well-studied combinatorial optimization problem is the MaxCut problem, where the aim is to find the partition of the graph's nodes into two complementary sets, such that the total weight of the edges between these two sets is as large as possible. MaxCut is equivalent to the Ising model given by Equation (2) with $h_i = 0$ and $J_{ij} = w_{ij}$, where w_{ij} is the weight of the edge between i th and j th nodes in the graph.

As a performance metric, we use the approximation ratio $\eta = E/E_{\text{opt}}$, where E is the energy of the found solution and E_{opt} is the energy of the optimal solution. To find the optimal solution, we use SCIP solver [36].

5.3.2 | Graph coloring

Proper coloring in a graph $G = (V, E)$ is an assignment of colors to the vertices in a way that no two adjacent vertices are assigned the same color. For a given number of colors, k , we consider the optimization problem that aims to minimize the amount of improperly colored edges. By introducing binary variables $x_{v,i}$, where $v \in V, i \in \{1, \dots, k\}$ and $x_{v,i}$ is equal to one if vertex v is assigned the color i , we can formulate the above problem as a QUBO problem [37]:

$$C(\vec{x}) := \lambda \sum_{v \in V} \left(1 - \sum_{i=1}^k x_{v,i} \right)^2 + \sum_{(v,w) \in E} \sum_{i=1}^k x_{v,i} x_{w,i} \rightarrow \min_{x_{v,i} \in \{0,1\}} \quad (15)$$

It's possible to decode a binary assignment in a coloring only if each vertex is assigned to strictly one color, i.e., $\sum_{i=1}^k x_{v,i} = 1$. The first term in Equation (15) with prefactor $\lambda > 0$ aims to penalize

the unfeasible solutions. If the graph can be colored in k colors, the optimal assignment $\vec{x}$ corresponds to a proper coloring and has $C(\vec{x}) = 0$.

6 | Results

This section presents the results obtained from the numerical simulation of our algorithm and its implementation on a QPU. We start with MaxCut problems. First, we employ algorithms based on bilinear relaxation (i.e., we set $r = 1$ and $l = n$), and then we show how to improve the performance using extra qubits to encode more groups. After that, we consider the graph coloring problem as an example of a constrained problem where encoding a larger problem-specific neighborhood is essential to getting a good solution. Finally, we run an experiment on a real IBM quantum device.

6.1 | Bilinear Relaxation

First, we investigate the performance of quantum algorithms based on bilinear relaxation. For this numerical experiment, we choose the MaxCut problem on two graph instances with 256 nodes: one is a 3-regular graph, and the other is a fully connected graph. In both cases, edge weights are sampled independently from a uniform distribution over $[-1, 1]$.

We benchmark our method with $r = 1$ against the minimal encoding algorithm from Refs. [8, 9]. Since $r = 1$ implies $l = n = 256$, our method requires 8 qubits, whereas the minimal encoding needs one additional ancilla qubit. For a fair comparison, in both cases, we optimize the same ansatz depicted in Figure 3 with L-BFGS-B using exact circuit simulation. For our method, we set the hyperparameter values: $\alpha = 2$, $M = 256$, $S = 1$, $R = 1$. We optimize the circuit with the number of layers ranging from 2 to 14.

To provide additional insights, we run the classical one-local search algorithm described in Section 5.2. We also plot the results obtained with a direct optimization of the bilinear relaxation (Equation (4)) with the L-BFGS-B method. To gather statistics, we repeat all the experiments 50 times as explained in Section 5. The results are presented in Figure 5.

This numerical experiment confirms that classical optimization of the bilinear relaxation given by Equation (4) provides solutions comparable to those obtained via one-local search, with the performance of the quantum algorithms plateauing approximately on the same level. Additionally, when the number of layers is small, our method, even with $r = 1$, outperforms the previously proposed quantum minimal encoding approach.

6.2 | Encoding Larger Neighborhood

We now demonstrate how the solutions can be improved by the use of extra qubits to encode a larger number of groups. For this experiment, we selected a three-regular MaxCut problem with $n = 512$ nodes. The edge weights were randomly sampled from a uniform distribution over $[-1, 1]$. Our algorithm was tested with

varying numbers of encoded groups. Since the problem is sparse, we take all connected groups G of size $\leq r$ for four different values $r = 1, 2, 3, 4$, as we explained in Section 2.3. The hyperparameters were fixed at $M = n = 512$, $\alpha = 7$, $S = n = 512$ and $R = 10$. The algorithm was launched from 48 random initial solutions $\vec{Z}_0$. For comparison, a classical local search was run within the same neighborhood from the same initial solutions.

To evaluate the results, the ensemble of final solutions from independent runs was compiled into an empirical cumulative distribution function (ECDF) of the objective function values. The ECDF for 16 layers is shown on the right side of Figure 6. On the left side, we show the average approximation ratio over all runs for the given hyperparameters.

The results indicate that, given a sufficiently deep circuit, increasing the number of groups improves the quality of the solutions. Notably, for small r , the quantum algorithm achieves solutions that are slightly superior to those obtained through classical local search within the corresponding neighborhood. We attribute this improvement to the multiple optimization rounds performed by the quantum algorithm.

To estimate the scaling of the required circuit depth, we repeat the experiment for a set of four three-regular graphs with $n = 64, 128, 256, 512$ nodes. For each problem and each r , we define the number of layers required to reach the same average approximation ratio as the corresponding classical local search. We compare the number of circuit parameters with the number of neighbors l . The results are presented in Figure 7.

Remarkably, the required number of variational parameters is much smaller than l , and the advantage becomes more evident with the growth of r . This highlights a key advantage of the quantum algorithm over the classical optimization of the auxiliary function. While the classical approach requires optimizing l parameters corresponding to the total number of neighbors, the quantum algorithm achieves comparable or better performance with significantly fewer variational parameters.

6.3 | Graph Coloring

Here, we present an example of a problem-specific approach to selecting groups for encoding, focusing on a constrained problem: the graph coloring problem.

The minimal encoding approach is unsuitable for this problem because flipping a single bit in any feasible solution results in an infeasible one. Consequently, all feasible solutions become local minima due to the penalty term introduced in Equation (15).

To make a neighborhood of one feasible solution contain another feasible solution, we choose such variables in the auxiliary function such that each of them flips a pair of bits $(x_{v,i}, x_{v,j})$. The set of such variables for all v and $i \neq j$ contains all groups that switch the vertex color. Note that such a choice is more efficient than taking all connected pairs since the latter contains pairs $(x_{v,i}, x_{w,i})$ that do not preserve feasibility.

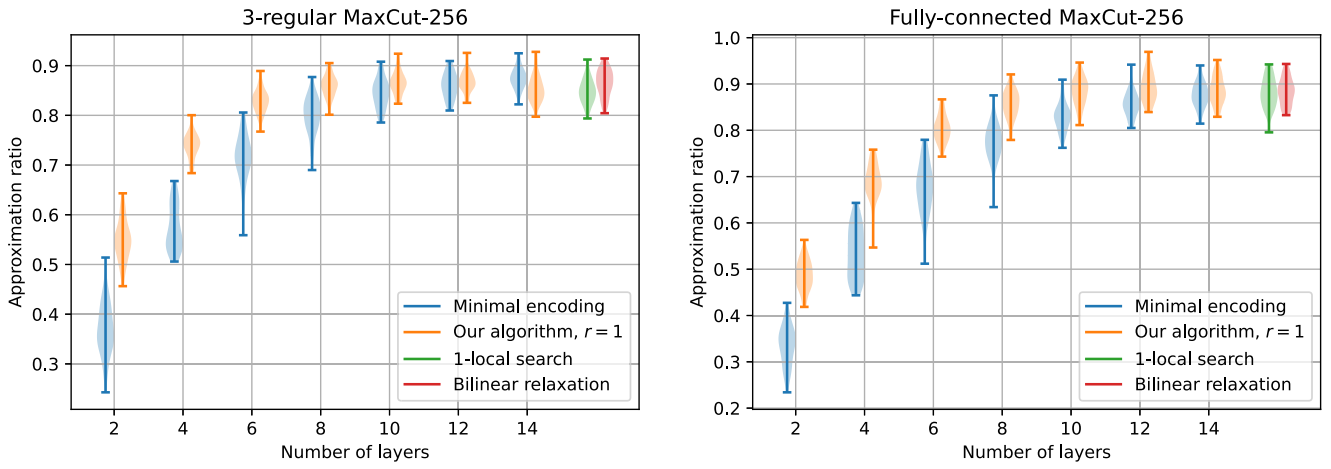


FIGURE 5 | Approximation ratio of solutions obtained by our algorithm with $r = 1$ in comparison to minimal encoding [8, 9], one-local search and the direct optimization of the bilinear relaxation (Equation (4)) with the L-BFGS-B method.

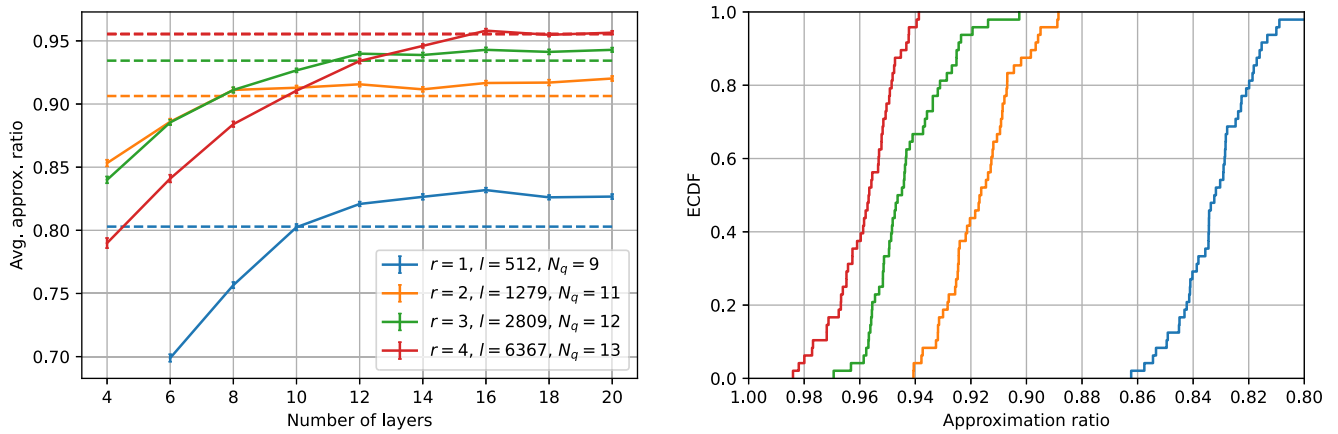


FIGURE 6 | Left: the average approximation ratio for MaxCut-512 found by the algorithm with different hyperparameter configurations: number of neighbors and number of layers. The dashed lines indicate the average approximation ratio of the corresponding (with the same neighborhood) classical local search started from the same initial solutions. Right: the empirical cumulative distribution function (ECDF) of the approximation ratios of the solutions found by the quantum algorithm with 16 layers.

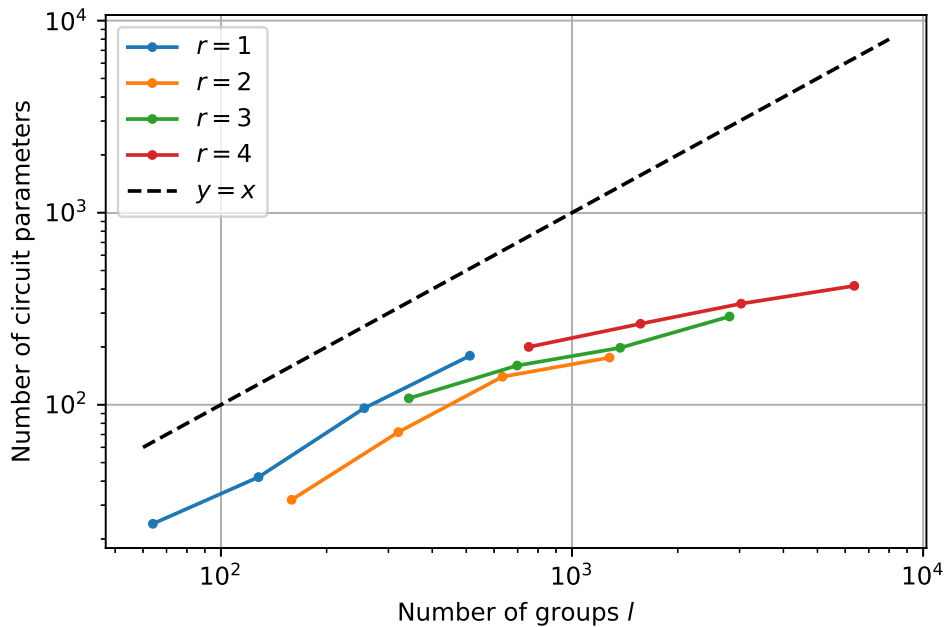


FIGURE 7 | Number of circuit parameters required for reaching the same average approximation ratio as classical local search.

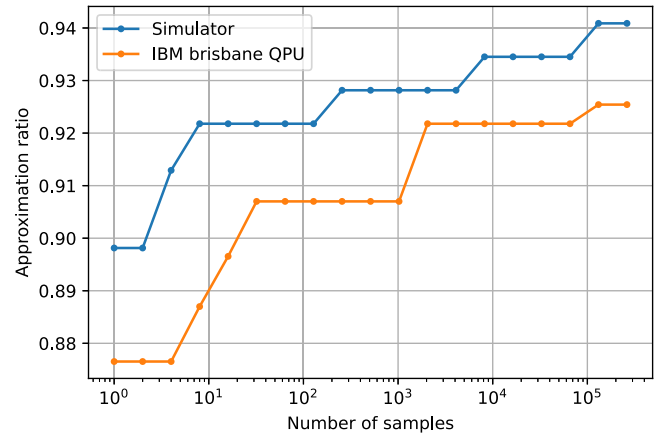
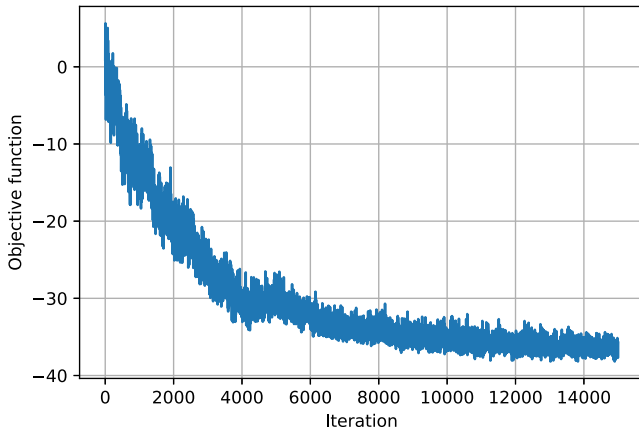


FIGURE 8 | QPU results for the MaxCut problem on a three-regular graph with 128 nodes, where the edge weights are randomly sampled from the interval $[-1, 1]$. The quantum circuit has seven qubits and eight layers. Left: Convergence of parameter optimization with SPSA algorithm over iterations using a numerical simulation that emulates the finite sampling of a quantum circuit with ideal gates. Right: approximation ratios of the best solution from the optimized circuit executed on the simulator and IBM brisbane QPU, plotted against the number of most probable samples drawn from the distribution generated by the flip probabilities.

We take the instance *myciel7* from the benchmark in Ref. [38] for the experiment. This graph, $G = (V, E)$, has $|V| = 191$ vertices, $|E| = 2360$ edges and $k = 8$ colors. Our experimental parameters are: $l = 5348$ groups, $N_q = 13$ qubits, $L = 20$ layers, $M = 1000$, $\alpha = 4$, $S = 10$, $R = 4$.

We repeat the experiment 100 times, each starting from a random feasible solution. In this setup, our algorithm successfully found the correct graph coloring in 19 out of 100 runs. To the best of our knowledge, this is the largest graph coloring instance solved with a quantum algorithm [37]. Note that algorithms based on the complete encoding, including QAOA, would require 1528 (logical) qubits for this instance, which is far from the capabilities of current quantum devices.

6.4 | Experiments on a QPU

Here, we test the applicability of the algorithm on a real noisy quantum device. To begin, we implement the variational optimization of the auxiliary cost function for solving a given MaxCut problem on a numerical simulator that emulates a quantum circuit with ideal gates subject to a finite number of measurements. Then, we test the obtained parameters on a real noisy IBM QPU and compare the outcome with the results from the numerical simulator.

For the experiment, we chose a MaxCut problem on a three-regular graph with 128 nodes, where the edge weights are randomly sampled from the interval $[-1, 1]$. The algorithm is executed with hyperparameters $r = 1$, $L = 8$, $M = 128$, $\alpha = 2$ and $R = 1$. The number of required qubits for this experiment is $N_q = 7$. The circuit optimization is performed using the Simultaneous Perturbation Stochastic Approximation (SPSA) algorithm [39] on a numerical simulator with ideal gates and finite measurement sampling. Specifically, the probability distribution, $\vec{P}$, is approximated by sampling $N = 1000$ shots from the exact state vector. The convergence plot of over 15,000 iterations, shown on the left of Figure 8, illustrates the optimization process under

these conditions.

Following this, the optimized circuit is executed on the simulator and the IBM brisbane QPU [40] with $N = 10,000$ shots to estimate $\vec{q}$ and extract the most probable solutions (see Section 3.4). The best solutions found for varying numbers of the most probable samples are depicted on the right of Figure 8. As expected, the solutions sampled from the QPU are inferior to those from the simulator due to noise. However, our approach of sampling multiple probable solutions from $\vec{q}$ drastically improves the quality of the solutions found with a noisy QPU: while QPU-induced noise distorts $\vec{q}$, taking only the most probable solution would yield a significantly worse result compared to the simulator. By instead leveraging multiple high-probability solutions from the distribution, we partially mitigate the effects of the noise and achieve solutions that are closer to those generated by the simulator.

7 | Conclusion

In this paper, we introduced a variational quantum algorithm tailored for solving combinatorial optimization problems, offering a flexible encoding scheme that can utilize a variable number of qubits, ranging from the logarithm of the number of classical variables up to the number of classical variables itself. This adaptability bridges the gap between minimal and complete encodings, addressing the trade-off in near-term quantum computing, where the minimal encoding often yields suboptimal solutions, while the complete encoding demands an impractical number of qubits. Through analytical insights and numerical experiments, we demonstrated that our algorithm achieves comparable performance to classical local search within a defined neighborhood while potentially offering an advantage to explore a classically intractable set of neighbors.

We implement our algorithm for various problems where encoding a large neighborhood is essential for achieving competitive solutions. Our approach successfully tackles problems that are

currently beyond the reach of other quantum algorithms such as minimal encoding, QAOA, and VQE.

Several important directions remain open for future work. First, a systematic investigation of barren plateaus in the proposed framework is needed, particularly how the landscape of the auxiliary objective and its neighborhood size affect gradient concentration as the number of qubits increases. Second, while we employ a generic variational ansatz, problem-specific, and structure-aware ansätze may significantly improve trainability and performance by better encoding the geometry of the underlying optimization problem. Quantum Architecture Search techniques [41–43] – which automatically explore and optimize variational circuit structures for specific tasks – could help generate tailored, problem-aware ansätze that improve trainability and performance beyond generic templates by balancing expressivity, noise resilience, and hardware constraints. In addition, incorporating advanced strategies from classical metaheuristics, such as adaptive neighborhood selection, variable-depth search, or hybridization with tabu search, may further enhance performance within the hybrid quantum–classical paradigm.

Overall, our research offers a promising pathway for advancing quantum optimization algorithms on near-term devices, with the potential to solve more complex real-world problems as quantum hardware continues to evolve.

Acknowledgements

The authors thank Katsiaryna Tsarova and Dr. Chris Mansell for valuable discussions and suggestions.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

Data sharing is not applicable to this article as no new data were created or analyzed in this study.

References

1. B. Alidaee, H. Wang, and L. S. Sua, “An Efficient Closed-Form Formula for Evaluating R-Flip Moves in Quadratic Unconstrained Binary Optimization,” *Algorithms* 16, no. 12 (2023): 557.
2. S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by Simulated Annealing,” *Science* 220, no. 4598 (1983): 671–680.
3. G. Palubeckis, “Multistart Tabu Search Strategies for the Unconstrained Binary Quadratic Optimization Problem,” *Annals of Operations Research* 131 (2004): 259–282.
4. P. Merz and K. Katayama, “Memetic Algorithms for the Unconstrained Binary Quadratic Programming Problem,” *Bio Systems* 78 (2005): 99–118.
5. Y. Wang and J.-K. Hao, *Metaheuristic Algorithms* (Springer International Publishing, 2022): 241–259.
6. T. Tomesh, Z. H. Saleem, and M. Suchara, “Quantum Local Search with the Quantum Alternating Operator Ansatz,” *Quantum* 6 (2022): 781.
7. C. Y. Liu and H. S. Goan, “Reinforcement Learning Quantum Local Search,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2023), 246–247, <https://doi.org/10.1109/QCE57702.2023.10226>.
8. B. Tan, M. A. Lemonde, S. Thanasilp, J. Tangpanitanon, and D. G. Angelakis, “Qubit-Efficient Encoding Schemes for Binary Optimisation Problems,” *Quantum* 5 (2021): 454.
9. M. R. Perelshtein, A. I. Pakhomchik, A. A. Melnikov, et al., “NISQ-Compatible Approximate Quantum Algorithm for Unconstrained and Constrained Discrete Optimization,” *Quantum* 7 (2023): 1186.
10. Y. Tene-Cohen, T. Kelman, O. Lev, and A. Makmal, “A Variational Qubit-Efficient MaxCut Heuristic Algorithm,” (2023), <https://arxiv.org/abs/2308.10383>.
11. Y. Chatterjee, E. Bourreau, and M. J. Rančić, “Solving Various NP-Hard Problems Using Exponentially Fewer Qubits on a Quantum Computer,” *Physical Review A* 109 (2024): 052441.
12. M. J. Rančić, “Noisy Intermediate-Scale Quantum Computing Algorithm for Solving an n -Vertex MaxCut Problem with $\log(n)$ Qubits,” *Physical Review Research* 5 (2023): L012021.
13. R. Storn and K. Price, “Differential Evolution - A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces,” *Journal of Global Optimization* 11 (1997): 341–359.
14. A. Peruzzo, J. McClean, P. Shadbolt, et al., “A Variational Eigenvalue Solver on a Photonic Quantum Processor,” *Nature Communications* 5 (2014): 4213.
15. P. Diez-Valle, D. Porras, and J. J. García-Ripoll, “Quantum Variational Optimization: The Role of Entanglement and Problem Hardness,” *Physical Review A* 104, no. 6 (2021): 062426.
16. E. Farhi, J. Goldstone, and S. Gutmann, “A Quantum Approximate Optimization Algorithm,” (2014).
17. K. Blekos, D. Brand, A. Ceschini, et al., “A Review on Quantum Approximate Optimization Algorithm and Its Variants,” *Physics Reports* 1068 (2024): 1–66.
18. B. Fuller, C. Hadfield, J. R. Glick, et al., “Approximate Solutions of Combinatorial Problems via Quantum Relaxations,” *IEEE Transactions on Quantum Engineering* 5, no. 01 (2024): 1–15.
19. T. Patti, J. Kossaifi, A. Anandkumar, and S. Yelin, “Variational Quantum Optimization with Multibasis Encodings,” *Physical Review Research* 4 (2022).
20. B. Sundar and M. Dupont, “Qubit-Efficient Quantum Combinatorial Optimization Solver,” (2024), <https://arxiv.org/abs/2407.15539>.
21. M. Sciorilli, L. Borges, T. L. Patti, et al., “Towards Large-Scale Quantum Optimization Solvers with Few Qubits,” *Nature Communications* 16, no. 1 (2025).
22. F. Glover, G. Kochenberger, and Y. Du, “Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models,” *4OR* 17 (2019).
23. A. Lucas, “Ising Formulations of Many NP Problems,” *Frontiers in Physics* 2 (2014): 5.
24. E. Boros and P. L. Hammer, “Pseudo-Boolean Optimization,” *Discrete Applied Mathematics* 123 (2002): 155–225.
25. C. Laneve, T. A. Lascu, and V. Sordoni, “The Interval Analysis of Multilinear Expressions,” *Electronic Notes in Theoretical Computer Science* 267, no. 2 (2010): 43–53.
26. J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, “Barren Plateaus in Quantum Neural Network Training Landscapes,” *Nature Communications* 9, no. 1 (2018).
27. R. P. Brent and P. Zimmermann, *Modern Computer Arithmetic* (Cambridge University Press, 2010).
28. D. Welsh, *Approximate Counting* (Cambridge University Press, 1997): 287–324.
29. L. Ann and G. Jerrum, “Counting Unlabelled Subtrees of a Tree Is #P-Complete,” *LMS Journal of Computation and Mathematics* 3 (2000).
30. L. Leone, S. F. Oliviero, L. Cincio, and M. Cerezo, “On the Practical Usefulness of the Hardware Efficient Ansatz,” *Quantum* 8 (2024): 1395.

31. M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, "Evaluating Analytic Gradients on Quantum Hardware," *Physical Review A* 99 (2019): 032331.
32. M. Larocca, S. Thanasilp, S. Wang, et al., "A Review of Barren Plateaus in Variational Quantum Computing," 2024, <https://arxiv.org/abs/2405.00781>.
33. J. Cunningham and J. Zhuang, "Investigating and Mitigating Barren Plateaus in Variational Quantum Circuits: A Survey," *Quantum Information Processing* 24, no. 2 (2025).
34. S. X. Zhang, J. Allcock, Z. Q. Wan, et al., "TensorCircuit: A Quantum Software Framework for the NISQ Era," *Quantum* 7 (2023): 912.
35. R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, "A Limited Memory Algorithm for Bound Constrained Optimization," *SIAM Journal on Scientific Computing* 16 (1995): 1190–1208.
36. S. Bolusani, M. Besançon, K. Bestuzheva, et al., "The SCIP Optimization Suite 9.0, Optimization Online," (Technical Report, 2024), <https://optimization-online.org/2024/02/the-scip-optimization-suite-9-0/>.
37. Z. Tabi, K. H. El-Safty, Z. Kallus, et al., "Quantum Optimization for the Graph Coloring Problem with Space-Efficient Embedding," in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2020): 56–62.
38. M. Trick, Graph Coloring Instances, <https://mat.gsia.cmu.edu/COLOR/instances.html>.
39. J. C. Spall, "An Overview of the Simultaneous Perturbation Method for Efficient Optimization," *Johns Hopkins Apl Technical Digest* 19 (1998): 482–492.
40. "IBM Quantum," (2024), accessed: 2024-12-12. <https://quantum.ibm.com/>.
41. S. X. Zhang, C. Y. Hsieh, S. Zhang, and H. Yao, "Differentiable Quantum Architecture Search," *Quantum Science and Technology* 7, no. 4 (2022): 045023.
42. Y. Du, T. Huang, S. You, M. H. Hsieh, and D. Tao, "Quantum Circuit Architecture Search for Variational Quantum Algorithms," *npj Quantum Information* 8, no. 1 (2022).
43. D. Martyniuk, J. Jung, and A. Paschke, "Quantum Architecture Search: A Survey," in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2024): 1695–1706, <https://doi.org/10.1109/QCE60285.2024.00198>.
44. G. G. Guerreschi and M. Smelyanskiy, "Practical Optimization for Hybrid Quantum-Classical Algorithms," (2017), <https://arxiv.org/abs/1701.01450>.
45. C. Bybee, D. Kleyko, D. E. Nikonov, A. Khosrowshahi, B. A. Olshausen, and F. T. Sommer, "Efficient Optimization with Higher-Order Ising Machines," (2022).
46. A. Genitrini and M. Pépin, "Lexicographic Unranking of Combinations Revisited," *Algorithms* 14, no. 3 (2021): 97.

Appendix A: Comparison With Complete Encoding

A.1 | Complete Encoding Review

Let us briefly review the complete encoding in the context of combinatorial optimization. This encoding requires a number of qubits equal to the number of classical variables: $N_q = n$. The objective is to minimize the average energy:

$$\langle E \rangle = \langle \psi | H | \psi \rangle \quad (\text{A1})$$

where H is the Ising Hamiltonian. Since it is diagonal in the computational basis, Equation (A1) can be rewritten as the average energy over a probability distribution over all possible states. It's convenient to use QUBO notation here since the bitstring, $\vec{\mu}$, which denotes the quantum

state matches the corresponding solution $\vec{x}$.

$$\langle C \rangle = \sum_{\mu} P_{\mu} C(\vec{\mu}) \quad (\text{A2})$$

where $C(\vec{\mu})$ is the QUBO objective function (Equation (1)) and $P_{\mu} = |\langle \mu | \psi \rangle|^2$. The estimate of Equation (A2), obtained by sampling the quantum circuit N times, is

$$\langle \hat{C} \rangle = \frac{1}{N} \sum_{j=1}^N C(\vec{\mu}^{(j)}) \quad (\text{A3})$$

where $\vec{\mu}^{(j)}$ is the j -th measurement outcome. The estimate is unbiased, and its variance decreases $\propto 1/N$ [44]. The MSE is equal to the variance since the estimate is unbiased. Note that it doesn't explicitly depend on the number of qubits.

A.2 | Comparison With our Encoding

To see the similarity with our method, consider any solution, $\vec{x}$, as a result of flipping the group $G = \{i : x_i = 1\}$ in an initial zero bitstring. For complete encoding, the value of P_{μ} then represents "flipping probability," where the flip of only one group is allowed. In our auxiliary function, each group is independently flipped with probability $p_{\mu} = (1 - q_{\mu})/2$. We can make our encoding quite similar to the complete one as follows:

1. Encode all the subsets of the spins (including the empty set). Then, the total number of groups is 2^n .
2. Use the mapping $G_{\mu} = \{i : \mu_i = 1\}$. In this case, there is a one-to-one correspondence between a qubit and a spin, just as with complete encoding.
3. Choose $\vec{x}^{(0)} = \vec{0}$ ($\vec{Z}^{(0)} = \vec{1}$) as initial solution. In that case, flipping $G_{\mu} = \{i : \mu_i = 1\}$ in $\vec{x}^{(0)}$ produces the solution $\vec{\mu}$. Now the outcome μ is related to the solution $\vec{\mu}$, as in complete encoding.
4. Choose a small $1 \lesssim M \lesssim 2$. We thereby allow no more than one negative q_{μ} , ensuring that the most probable solution differs by at most one group flip from the initial $\vec{x}^{(0)} = \vec{0}$. We emphasize that the auxiliary function is still the average energy of the probability distribution over all (degenerate) $2^l = 2^{2^n}$ states $\vec{z}$, where each group is either flipped or not. However, the small M provides a low probability of multiple flips. In addition, as shown in Figure 4 in the main text for $M = 2$, the estimation error decreases starting from a few shots, which makes the estimation no harder than for complete encoding.

A.3 | Numerical Experiment

We benchmark our algorithm against VQE by taking a fully connected random Ising instance of size $n = 20$. We configure our algorithm according to the following scheme: we encode all subsets ($r = 20$), set $\vec{Z}^{(0)} = \vec{1}$, $M = 2$ and $\alpha = 2$. We take 100 random initial solutions for each number of layers and perform $R = 1$ round of optimization for each of them on a noiseless simulator. For comparison, we run VQE with the same ansatz and our algorithm with $r = 1$, $M = 20$, $\alpha = 2$ and $r = 2$, $M = 40$, $\alpha = 3$. For VQE, after optimization, we take 500 samples and choose the best one. For our algorithm, we perform one round ($R = 1$), take $N = 500$ quantum circuit samples and the $S = 500$ most probable solutions. Such a choice is fair in that for both VQE and our algorithm, the circuit is sampled 500 times and the objective function is evaluated for 500 solutions. For each number of layers, we plot the average approximation ratio of the solutions obtained from the ensemble of 100 initial points. For the best number of layers, we plot the entire empirical cumulative distribution function. The results are presented in Figure A1.

The experiment demonstrates that the performance of our algorithm with $r = 20$ is comparable to that of VQE. However, we also show in Figure A1 that using 20 qubits is inefficient, as our algorithm with $r = 1$ and $r = 2$, requiring only five and eight qubits, respectively, provides better results.

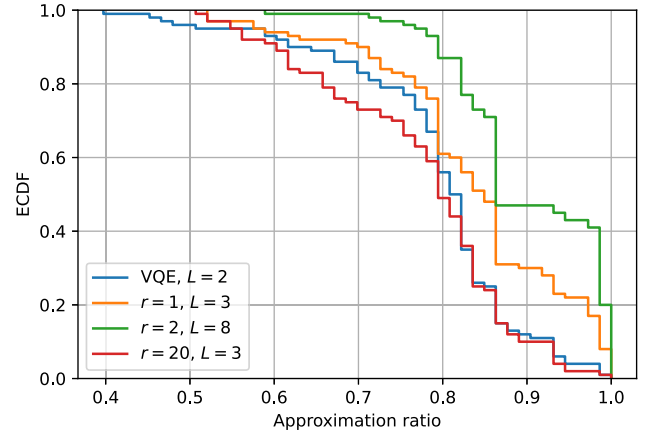
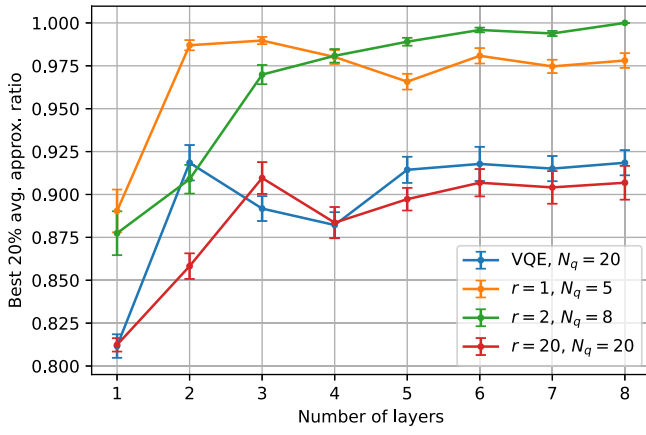


FIGURE A1 | Left: the average approximation ratio of the top 20% of solutions for Ising-20 found by VQE and our algorithm with different values of r and different numbers of layers. Right: the empirical cumulative distribution function (ECDF) of the approximation ratios of the solutions found by the algorithms with the best number of layers.

The poor performance of VQE and our algorithm with $r = 20$ can be attributed to the following: both the auxiliary function (Equation (9)) and the VQE average energy (Equation (A2)) involve 2^{20} variables (for VQE we consider P_μ here). If these functions were optimized directly, the optimal solution would always be found, as all local minima map to the global minimum of the original problem. However, in the experiment we conducted, the quantum algorithms optimize only up to 320 parameters, leading to numerous local minima induced by the hardware-efficient ansatz [32], significantly deteriorating the quality of the solutions.

In contrast, the auxiliary function for $r = 1$ and $r = 2$ contains only 20 and 210 variables, respectively, but has local minima corresponding to r -local search. Here, the performance is constrained by the limitations of local search itself. Notably, the performance is also not very good – the optimal solution to such a small problem is found in no more than 20% of the runs. This limitation highlights why pure local search is seldom employed; instead, more advanced metaheuristics [5] that build upon local search are typically used to overcome these challenges. Our quantum algorithm can also be integrated into such sophisticated metaheuristics, providing a promising direction for future research to enhance solution quality.

Appendix B: Extension to Higher-Order Problems

Here, we show how to extend our method to the PUBO problem, which is equivalent to a generalized Ising model with higher-order spin interactions [45]. Let n be the number of variables, $V = \{1, 2, \dots, n\}$. Then, the energy can be written as:

$$E = \sum_{S \subseteq V} J_S \prod_{i \in S} Z_i \quad (\text{B1})$$

with $J_S \in \mathbb{R}$ the model parameters. Substituting Equation (6) from the main text into Equation (B1), we obtain:

$$E = \sum_{S \subseteq V} J_S \prod_{i \in S} Z_i^{(0)} \prod_{k: i \in G_k} z_k \quad (\text{B2})$$

The identity $z_k^2 = 1$ allows us to reduce the energy (Equation (B2)) to a multilinear form and then replace the discrete $z_k \in \{-1, 1\}$ with continuous $q_k \in [-1, 1]$. This results in an auxiliary function, $\mathcal{E}(\vec{q})$, that can be optimized with a variational quantum algorithm according to the same scheme as for the second-degree problem discussed in the main text.

Appendix C: Efficient Trivial Mapping

Suppose the groups are ordered in the following sequence:

$$\{0\}, \{1\}, \{2\}, \dots, \{n-1\}, \{0, 1\}, \{0, 2\}, \dots, \{0, n-1\}, \{1, 2\}, \dots, \{n-2, n-1\}, \{0, 1, 2\}, \{0, 1, 3\}, \dots$$

Here, we outline an efficient algorithm for determining the group, G , corresponding to a given measurement output, μ . This procedure enables the implementation of the trivial mapping described in Section 3.1 for a large number of groups where storing all groups explicitly becomes infeasible. The algorithm is a modified version of the lexicographic unranking method for combinations [46].

1. **Determine the Size of the Subset:** Let $C(k) = \sum_{i=1}^k \binom{n}{i}$, where $\binom{n}{i}$ is a binomial coefficient. The value $C(k)$ represents the total number of subsets of size $\leq k$.

Find the size, m , of the subset, G , such that:

$$C(m-1) < \mu \leq C(m)$$

Compute the adjusted position within subsets of size m as:

$$\mu' = \mu - C(m-1)$$

2. **Generate the μ' -th Subset:** Start with an empty subset, G , and construct the μ' -th subset of size m :
 - a. Initialize $m_0 = m$ and $\mu_0 = \mu'$.
 - b. For each candidate element, x , from 0 to $n-1$, do the following:
 - Compute the number of subsets of size m_0 that can be formed with x as the smallest element:

$$\text{count} = \binom{n-x-1}{m_0-1}$$

- If $\mu_0 \leq \text{count}$, add x to G and decrement m_0 by 1. Otherwise, subtract the count from μ_0 .
- c. Repeat until $m_0 = 0$.

This procedure requires $O(n)$ evaluations of large binomial coefficients. The largest possible binomial coefficient is $\binom{n}{r}$, which has an evaluation complexity $O[rM(d)]$, where $M(d) \leq O(d^2)$ is the complexity of d -bit number multiplication and $d \leq \log_2 n^r$. Therefore, the overall complexity of the procedure is $O[nrM(r \log n)]$.

ALGORITHM 1 | Sampling the Most Probable Configurations.

```

1:   Input: spin probabilities  $\vec{p}$ , number of samples  $S$ 
2:   Compute  $g_k$  for each spin:  $g_k \leftarrow p_k/(1 - p_k)$  if  $p_k < 0.5$ ,
      else  $(1 - p_k)/p_k$ 
3:   Compute the initial configuration:
       $\vec{z}^{(1)} \leftarrow 1 - 2 \cdot \text{ROUND}(\vec{p})$ 
4:   Compute the probability of  $\vec{z}^{(1)}$ :
       $P^{(1)} \leftarrow \prod_{k: z_k^{(1)} = -1} p_k \prod_{k: z_k^{(1)} = 1} (1 - p_k)$ 
5:   Initialize a priority list  $A \leftarrow [(\vec{z}^{(1)}, P^{(1)})]$ 
6:   for  $k = 1$  to  $\text{LENGTH}(\vec{p})$  do
7:       Create a new list  $B \leftarrow []$ 
8:       for  $(\vec{z}, P)$  in  $A$  do
9:           Flip the  $k$ -th spin:  $\hat{F}_k \vec{z}$ 
10:          Compute the new probability:  $P_{\text{new}} \leftarrow g_k \cdot P$ 
11:          Append  $(\hat{F}_k \vec{z}, P_{\text{new}})$  to  $B$ 
12:       end for
13:       Merge  $A$  and  $B$ :  $A \leftarrow A \cup B$ 
14:       Sort  $A$  by probability  $P$  in descending order
15:       Keep the top  $S$  configurations:  $A \leftarrow A[:S]$ 
16:   end for
17:   Return:  $A$ , containing the  $S$  most probable
      configurations and their probabilities

```

Appendix D: Sampling the Most Probable Solutions

We propose an efficient algorithm to determine the S most probable configurations $\vec{z}$ of a multivariate Bernoulli distribution with independent spins. The probability for the k -th spin to be down is defined as $p_k := P(z_k = -1)$. The probability of a given configuration, $\vec{z}$, is given by:

$$P(\vec{z}) = \prod_{k: z_k = -1} p_k \prod_{k: z_k = 1} (1 - p_k)$$

The goal is to compute the S most probable configurations, denoted as $\vec{z}^{(1)}, \vec{z}^{(2)}, \dots, \vec{z}^{(S)}$. The algorithm builds these solutions iteratively based on their probabilities.

The most probable configuration, $\vec{z}^{(1)}$, can be straightforwardly determined as:

$$z_k^{(1)} = \begin{cases} 1 & \text{if } p_k < 0.5, \\ -1 & \text{otherwise.} \end{cases}$$

Let $\hat{F}_k \vec{z}$ denote the configuration obtained by flipping the k -th spin in $\vec{z}$. If the k -th spin is in its most probable state, flipping it scales the probability of the configuration by a factor, g_k , defined as:

$$g_k = \frac{P(\hat{F}_k \vec{z})}{P(\vec{z})} = \begin{cases} p_k/(1 - p_k) & \text{if } p_k < 0.5, \\ (1 - p_k)/p_k & \text{if } p_k \geq 0.5. \end{cases}$$

Since $g_k \leq 1$, flipping a spin from its most probable state always reduces the overall probability of the configuration.

The algorithm for sampling the most probable configurations is given in Algorithm 1. It can be explained as follows:

1. **Initialization:** The algorithm starts with the most probable configuration $\vec{z}^{(1)}$ and its probability $P^{(1)}$. This configuration is stored in list A along with its probability.
2. **Iterative Expansion:** At each step, the algorithm generates all the configurations obtained by flipping one spin of each current configuration in A . The probabilities are updated using the precomputed g_k .
3. **Pruning:** To prevent exponential growth in the number of configurations, A is sorted by probability in descending order, and only the top S configurations are retained. Importantly, “child” configurations of the rejected ones – configurations that differ only in spins not yet considered for flipping – are automatically excluded. Since $g_k \leq 1$ for all k , they have an even lower probability, which ensures that we don’t lose any solutions of the interest.
4. **Termination:** After all the spins have been processed, A contains the S most probable configurations.

The most time-consuming procedure in the algorithm is sorting the array A containing at most $2S$ elements, which can be done in $O(S \log S)$. Therefore, the complexity of the algorithm is $O(lS \log S)$, where l is the length of $\vec{p}$. However, if the circuit is measured with a finite number of shots, N , there are no more than N nonzero p_k , reducing the complexity to $O(NS \log S)$.

Appendix E: Algorithm Pseudocode

Algorithm 2 presents the complete pseudocode for the introduced qubit-efficient quantum local search method. The algorithm takes Ising problem specification, hyperparameters, and quantum circuit configuration as inputs, and returns an optimized solution.

Require: Ising coefficients $\{h_i, J_{ij}\}$, neighborhood parameter r , hyperparameters $M, \alpha, R, S, N_{\text{shots}}, L$

Ensure: Best solution $\vec{Z}^*$ found

```

1:      procedure QuantumLocalSearch
2:       $\vec{Z}^{(0)} \leftarrow$  random initial solution  $\triangleright$  Section 3.4
3:       $\vec{Z}^* \leftarrow \vec{Z}^{(0)}, E^* \leftarrow E(\vec{Z}^{(0)})$   $\triangleright$  Initialize best solution
4:      Construct groups  $\{G_\mu\}$  for  $r$ -local search  $\triangleright$  Section 2.3
5:      Choose mapping and the number of qubits  $N_q$   $\triangleright$ 
        Section 3.1
6:      Initialize quantum circuit ansatz with  $N_q$  qubits and  $L$ 
        layers  $\triangleright$  Section 3.3, Figure 3
7:      for round  $\leftarrow 1$  to  $R$  do
8:       $\vec{\theta} \leftarrow$  random initialization in  $[0, 2\pi)$ 
9:      converged  $\leftarrow$  False
10:     while not converged do
11:     Estimate  $\vec{P}(\vec{\theta})$  using  $N_{\text{shots}}$  measurements
12:     Compute  $\vec{q}$  from  $\vec{P}$  via Equation (11)  $\triangleright$  Section 3.2
13:     Evaluate  $\mathcal{E}(\vec{q})$  using Equation (9)
14:     if using gradient-based optimization then
15:     Compute gradient  $\frac{\partial \mathcal{E}}{\partial \vec{\theta}}$  via parameter-shift rule  $\triangleright$ 
        Equation(13)
16:     Update  $\vec{\theta}$  using gradient-based optimizer
17:     else
18:     Update  $\vec{\theta}$  using gradient-free optimizer
19:     end if
20:     Check convergence criteria  $\triangleright$  e.g., gradient norm,
        function tolerance
21:     end while
22:     Obtain final  $\vec{q}$  using  $N_{\text{shots}}$  measurements
23:     Compute flip probabilities  $\vec{p} = (1 - \vec{q})/2$ 
24:     Sample  $S$  most probable  $\vec{z}$  from Bernoulli ( $\vec{p}$ )  $\triangleright$ 
        Algorithm 1, Appendix D
25:     Select  $\vec{z}^{\text{best}} = \arg \min_{\vec{z}} E(\vec{z})$   $\triangleright$  Using Equation(7)
26:     Decode  $\vec{Z}^{\text{best}} = \vec{Z}^{(0)} \circ \prod_{k: i \in G_k} z_k^{\text{best}}$   $\triangleright$  Equation(6)
27:     if  $E(\vec{Z}^{\text{best}}) < E^*$  then
28:      $\vec{Z}^* \leftarrow \vec{Z}^{\text{best}}, E^* \leftarrow E(\vec{Z}^{\text{best}})$ 
29:      $\vec{Z}^{(0)} \leftarrow \vec{Z}^{\text{best}}$   $\triangleright$  Restart with improved solution
30:     else
31:     break  $\triangleright$  Early stopping if no improvement
32:     end if
33:     end for
34:     return  $\vec{Z}^*, E^*$ 
35:     end procedure

```